Teaching a LLM to Count.

Generative AI

Alexis Tamayo

Udacity - Woolf University.

Master's degree in AI.

Project Overview

This project focuses on improving a large language model's ability to perform **precise procedural reasoning** through a controlled task: counting the number of occurrences of a letter in a word.

While modern LLMs are highly fluent, they often fail at tasks requiring **step-by-step logical accuracy**. This project addresses that limitation by training a model to explicitly reason through each step of the counting process.

The solution uses:

- **Qwen/Qwen2.5-3B-Instruct** as the base model
- **LoRA (Low-Rank Adaptation)** for efficient fine-tuning
- **GRPO (Group Relative Policy Optimization)** for reinforcement learning

The final goal is to produce a model that reliably:

1. Breaks down the word letter by letter
2. Tracks a running count
3. Produces a correct and structured final answer

Dataset Description

The dataset is a **synthetic dataset** specifically designed for the reasoning task.

- Built from a curated list of words (e.g., *banana, committee, mississippi*)
- Each word generates multiple training examples (one per unique letter)
- Total records generated: **~94 samples**

Each record includes:

- Input word
- Target letter
- Correct answer
- Natural language prompt

This design ensures:

- Full control over correctness
- Consistent formatting
- High-quality supervision for reasoning

Data Preparation & Exploration

The data pipeline includes:

1. Defining a base word list
2. Expanding into letter-specific records
3. Converting into a Hugging Face dataset
4. Formatting into chat-style prompts

Each example includes:

- A **system message** (reasoning instructions)
- A **user query** (counting task)

Exploration confirmed:

- Balanced variety of words
- Presence of repeated letters
- Clear mapping between input and expected output

Model Design

Base Model

- `Qwen/Qwen2.5-3B-Instruct`

Fine-tuning Strategy

- **LoRA (rank = 64)** to reduce memory cost
- Target modules:
  - Attention layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`)
  - MLP layers (`gate_proj`, `up_proj`, `down_proj`)

Design Rationale

- LoRA avoids full retraining of 3B parameters
- Rank 64 balances performance vs efficiency
- Targeting both attention and MLP layers improves reasoning adaptation

Training Process

Training follows a structured pipeline:

1. Baseline prompt evaluation
2. Improved Chain-of-Thought prompting
3. Reward function engineering
4. GRPO training (short + extended runs)

Reward Functions

- Numbering correctness
- Letter-by-letter spelling
- Running count accuracy
- Output format compliance
- Final answer correctness

Key Hyperparameters

- Learning rate: `1e-5`
- Batch size: `16`
- Generations: `4`
- GRPO beta: `1e-4`

Full training requires GPU (CUDA environment), not available locally

Model Evaluation

Evaluation includes:

1. Baseline Model

- Weak prompt → inconsistent results

2. Improved Prompt

- Chain-of-thought improves reasoning

3. Post-Training Comparison

- Original vs fine-tuned model
- Catastrophic forgetting check

Evaluation focuses on:

- Accuracy
- Reasoning quality
- Output structure

Results & Interpretation

Verified Outputs

Reward functions behaved correctly:

- Numbering: `[2.0, -1.5]`
- Spelling: `[2.0, 0.1]`
- Counting: `[1.0, -0.5]`
- Format: `[1.0, 0.0]`
- Final answer: `[2.0, -1.0]`

Interpretation

- Reward system successfully differentiates good vs bad outputs
- Model is ready for reinforcement learning
- Full performance depends on GPU training

Non-Technical Explanation

This project teaches an AI to **think more carefully instead of guessing**.

Instead of answering quickly, the model:

1. Reads each letter
2. Checks if it matches
3. Keeps track of the count
4. Gives a clear final answer

This improves reliability in tasks where **precision matters**.

Experimental Design Justification

Why Synthetic Data?

- Fully controlled
- Automatically labeled
- Perfect for structured reasoning

Why LoRA?

- Efficient
- Reduces compute cost
- Keeps base model intact

Why GRPO?

- Ideal for reasoning tasks
- Uses reward signals instead of labels

Why Multiple Rewards?

- Encourages correct reasoning steps
- Prevents shortcut learning
- Improves intermediate behavior

Workflow Completeness

The project is complete at the design level:

- Dataset created
- Prompts designed
- Reward functions implemented
- Training pipeline configured
- Evaluation methods defined

Remaining:

- Full GPU training execution

Bias and Risk Awareness

Key Risks

1. **Reward hacking** (model gaming rewards)
2. **Overfitting to format**
3. **Limited generalization**
4. **Environment limitations**

Mitigation

- Multiple reward functions
- Structured evaluation
- Separation of training vs testing

Future Improvements & Integration

1. Expand dataset with harder cases
2. Add validation/test split
3. Run full GPU training
4. Track accuracy metrics
5. Deploy model via API
6. Extend to similar reasoning tasks

Conclusion

This project demonstrates how to improve LLM reasoning using:

- Structured prompting
- Synthetic datasets
- Reinforcement learning
- Efficient fine-tuning (LoRA)

The system successfully prepares a model to perform **reliable step-by-step reasoning**, with final performance dependent on GPU-based training.

Reproducibility

Setup

```
Clone repo: https://github.com/atamayop2018/gen-AI2
uv python install 3.12.3
uv venv --python 3.12.3 .venv
uv pip install --python .venv/bin/python -r requirements.txt torch
```

Run

```
source .venv/bin/activate
jupyter lab
```

Open:

`gen_ai_fundamentals_project_starter.ipynb`

Requirement

- CUDA-enabled GPU for full training

References

- Qwen2.5 Model Documentation
- LoRA (Hu et al.)
- Hugging Face TRL (GRPO)
- Hugging Face PEFT
- Unsloth Documentation
- vLLM Documentation
- Udacity AI Nanodegree materials